



CIL-- 합성을 통한 Sparrow 분석기 공격

정원준

2026년 7월 10일

ROPAS@SNU

목차

Sparrow 분석기 공격 목표

CIL-- 언어

실행의미를 Big-Step으로 정의

Sparrow의 실행의미와 대응시키기

메모리를 포함하여 합성하는 방법

앞으로 할 일

Sparrow 분석기 공격 목표

Sparrow 공격 목표

■ completeness 공격:

abstract semantics가 concrete semantics를 너무 크게 포함함
(concrete = abstract인 그림) (concrete $\subset$ abstract인 그림)
(abstract = top인 그림)

Sparrow 공격 목표

■ completeness 공격:

abstract semantics가 concrete semantics를 너무 크게 포함함
(concrete = abstract인 그림) (concrete $\subset$ abstract인 그림)
(abstract = top인 그림)

■ soundness 공격:

concrete semantics가 abstract semantics에 포함되지 않음(버그)
(concrete $\not\subset$ abstract인 그림)

Sparrow의 기본 구조

(C->CIL->CFG 의 예시 그림)

예시는

```
int main() { int x = 1; if(x == 1) x++; return x; }
```

정도로

CIL-- 언어

CIL 언어

CIL이 뭐냐, C의 부분집합 느낌. 분석을 쉽게 할 수 있는 언어
애매하거나 복잡한 구조 단순화, side effect 제거, 정규화

CIL-- 언어

completeness를 공격하기 위해서는 튜링 완전한 subset이기만 하면
충분(Rice Theorem 덕분) soundness를 공격하기 위해서는 많은
feature가 있을수록 좋음(버그 찾는거니까)

CIL- : 의미적으로 관심없는 것들 제거, 나중에도 확장할 생각 없음
(typedef, struct 등) CIL-- : 나중에 추가했으면 좋겠지만 일단
미니멀하게 가기 위해 제거 (포인터 + malloc/free, 배열, int 외 type)

실행의미를 Big-Step으로 정의

Big-Step 기본 구조

CompCert의 Clight big-step 의미구조가 있긴한데, 여기서는
CIL--에서의 의미구조를 직접 정의

expression은? 메모리를 보면서 정의 메모리는? stack(frame)과 heap
이 있는데, 아직 heap은 없음

call은? frame을 하나 넣고, arg랑 로컬 변수 할당, return하는거까지
표현 (그림 넣기)

Sparrow의 실행의미와 대응시키기

실행 의미가 잘 정의되는가?

C->CIL->CFG ->CIL-- 구조.

가장 큰 문제: sparrow는 concrete semantics를 정확히 정의하지 않음.

C: ISO C 표준이 있는데, formal하진 않음 CIL: 그런것도 없음.

Sparrow: 애도 아예 없음 공격기: CIL--에 대한 big-step tree랑, derivator가 있긴 함

공격기의 concrete 실행의미가 Sparrow의 abstract 실행의미에 포함되지 않더라도, soundness가 깨졌다! 라고 말할 수 있는가?
(여기에 그림 넣기)

C 언어의 비결정성 보기

각종 비결정성을 모두 제거해야 공격을 제대로 할 수 있음

C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int (*)[n+ +])	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	undefined behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때,	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	포인터 없음

C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int (*)[n+ +])	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	undefined behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때,	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	포인터 없음

이외에도 malloc 실패 여부, struct의 byte 패딩, signed 정수 overflow 등 여러 가지 고려해야 할 비결정성이 많음

메모리를 포함하여 합성하는 방법

메모리는 코드를 따라 자연스럽게 나오는 것이 아닌가?

완전한 프로그램은 그 자체로 메모리를 결정할 수 있음

```
int main() int x; int y; x = 1; y = x + 1; return x;
```

메모리는 코드를 따라 자연스럽게 나오는 것이 아닌가?

완전한 프로그램은 그 자체로 메모리를 결정할 수 있음

`int main() int x; int y; x = 1; y = x + 1; return x;` 반면 부분
프로그램은 앞뒤 메모리의 관계만 알 수 있음

(전) `y = x + 1;` (후)

메모리를 같이 합성하지 않으면

증명나무의 구조

메모리를 같이 합성하지 않으면

증명나무의 구조 ($x = 1$; 에 해당하는 증명나무 그림)

메모리를 같이 합성하지 않으면

증명나무의 구조 ($x = 1$; 에 해당하는 증명나무 그림) ($y = x + 1$; 에
해당하는 증명나무 그림)

메모리를 같이 합성하지 않으면

증명나무의 구조 ($x = 1$; 에 해당하는 증명나무 그림) ($y = x + 1$; 에
해당하는 증명나무 그림) (둘을 합쳐서 연결한 그림)

메모리를 같이 합성하지 않으면

증명나무의 구조 ($x = 1$; 에 해당하는 증명나무 그림) ($y = x + 1$; 에
해당하는 증명나무 그림) (둘을 합쳐서 연결한 그림) 어 되나?

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

```
(x = 3; loopif(x == 0) break; x = x - 1;;)
```

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

```
(x = 3; loopif(x == 0) break; x = x - 1;;)    (x = 5; loopif(x == 0) break; x
= x - 1;;)
```

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

```
(x = 3; loopif(x == 0) break; x = x - 1;;) (x = 5; loopif(x == 0) break; x  
= x - 1;;) 조립이 잘 안 됨!
```

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

(x = 3; loopif(x == 0) break; x = x - 1;;) (x = 5; loopif(x == 0) break; x = x - 1;;) 조립이 잘 안 됨! 후처리를 한다고 해도 무한히 돌거나 정지 문제를 해결해야 하는 딜레마에 빠짐

하지만 조건문, 반복문을 만나면...

```
(loopif(x == 0) break; x = x - 1;;)
```

(x = 3; loopif(x == 0) break; x = x - 1;;) (x = 5; loopif(x == 0) break; x = x - 1;;) 조립이 잘 안 됨! 후처리를 한다고 해도 무한히 돌거나 정지 문제를 해결해야 하는 딜레마에 빠짐 따라서 메모리는 나중에 계산하면 안 되고, 작은 증명나무를 만들 때 전후 메모리가 확실하게 정해져야 한다.

합성 시 메모리 관리

실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다. 특히 struct같은거 보면 적당히 포인터 + 1 연산같은걸로 주소 옮겨다니기 가능.

합성 시 메모리 관리

실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다. 특히 struct같은거 보면 적당히 포인터 + 1 연산같은걸로 주소 옮겨다니기 가능. CIL, 혹은 CIL--에서는 모든 변수가 외딴섬으로 존재해, 직접 그 변수를 통하지 않으면 해당 메모리 위치를 참조할 수 없게 한다.

합성 시 메모리 관리

실제 C 언어에서는 주소 연산을 통해 인접한 메모리를 마구 참조 가능하다. 특히 struct같은거 보면 적당히 포인터 + 1 연산같은걸로 주소 옮겨다니기 가능. CIL, 혹은 CIL--에서는 모든 변수가 외딴섬으로 존재해, 직접 그 변수를 통하지 않으면 해당 메모리 위치를 참조할 수 없게 한다. 따라서 어느 정도는 x mapsto 1, y mapsto 2 이런 구조로 메모리를 생각할 수 있다.

stack frame 관리

메모리에는 크게 heap, stack, global/static이 있다.

stack frame 관리

메모리에는 크게 heap, stack, global/static이 있다. 이 중 일단 stack만 보기로 하자. 다음 간단한 함수를 보자. `int f(int x) if(x == 0) return 0; return f(x - 1);`

stack frame 관리

메모리에는 크게 heap, stack, global/static이 있다. 이 중 일단 stack 만 보기로 하자. 다음 간단한 함수를 보자. `int f(int x) if(x == 0) return 0; return f(x - 1);` main에서 `f(2)`를 호출했을 때 증명나무는 대략 다음과 같은 모양이다. (증명나무)

그렇지만 모든 frame을 보기엔 너무 크다

마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다. (3개 스택이 쌓인 모습)

그렇지만 모든 frame을 보기엔 너무 크다

마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다. (3개 스택이 쌓인 모습) 그렇지만 모두가 알듯이, 중요한(당장 봐야 하는)건 top frame뿐이다.

그렇지만 모든 frame을 보기엔 너무 크다

마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다. (3개 스택이 쌓인 모습) 그렇지만 모두가 알듯이, 중요한(당장 봐야 하는)건 top frame뿐이다. (모든 frame을 들고다니는 증명나무 그림)

그렇지만 모든 frame을 보기엔 너무 크다

마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다. (3개 스택이 쌓인 모습) 그렇지만 모두가 알듯이, 중요한(당장 봐야 하는)건 top frame뿐이다. (top frame 만 들고다니는 증명나무 그림)

그렇지만 모든 frame을 보기엔 너무 크다

마지막 $f(0)$ 이 호출되었을 때, 메모리 상태는 다음과 같다. (3개 스택이 쌓인 모습) 그렇지만 모두가 알듯이, 중요한(당장 봐야 하는)건 top frame뿐이다. (top frame 만 들고다니는 증명나무 그림) 포인터를 인자로 받으면 이전 frame에 간섭이 가능하지만, 그것도 결국 따로 접근 가능한 location으로 두고, 조립할때 location을 연결하면 된다.

결국 메모리 관리는

이제 합성에 필요한 메모리는 다시 $x \mapsto 1$, $y \mapsto 2$ 꼴이므로,

결국 메모리 관리는

이제 합성에 필요한 메모리는 다시 $x \mapsto 1$, $y \mapsto 2$ 꼴이므로,
(가능한 변수 이름(id) 목록, 가능한 값 목록)만 있으면 유한하게
메모리를 만들 수 있다.

결국 메모리 관리

이제 합성에 필요한 메모리는 다시 $x \mapsto 1$, $y \mapsto 2$ 꼴이므로, (가능한 변수 이름(id) 목록, 가능한 값 목록)만 있으면 유한하게 메모리를 만들 수 있다. completeness 공격의 종료를 보장하기 위해서는 증명나무의 크기 증가와 함께 변수 이름, 가능한 값의 범위를 점차 넓혀가면 된다.

앞으로 할 일

앞으로 할 일

- 지금까지 한 일 syntax, semantics(big-step), interpreter(CIL-- 코드
- > big-step proof tree) 등등 기본적인거 구현 완료

앞으로 할 일

- 지금까지 한 일 syntax, semantics(big-step), interpreter(CIL-- 코드
-> big-step proof tree) 등등 기본적인거 구현 완료
- 앞으로 할 일 bottom-up 기반으로 합성 시작 각종 가지치기(pruning),
휴리스틱을 통한 시간 단축 실행 중 AI 가이드?

앞으로 할 일

- 지금까지 한 일 syntax, semantics(big-step), interpreter(CIL-- 코드
-> big-step proof tree) 등등 기본적인거 구현 완료
- 앞으로 할 일 bottom-up 기반으로 합성 시작 각종 가지치기(pruning),
휴리스틱을 통한 시간 단축 실행 중 AI 가이드?
- 걱정거리 걱정거리: simple.c만 해도 크기가 매우매우 큼. (그림 삽입)
재귀적으로 합성할 것/아닌 것을 구분해서 쉬운 step은 바로바로
진행되게끔 하는 스킬이 필요할 듯

감사합니다